Reviewer Pack — Minimal Deligne-Lusztig Tree Depth and Resolution Proof Widt…

Entry 43e22ea23a6f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 12:49:12 UTC

Minimal Deligne-Lusztig Tree Depth and Resolution Proof Width

Entry ID: `43e22ea23a6f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 12:49:12 UTC

1. Conjecture

Field A (mathematical branch): Geometric Representation Theory (Deligne-Lusztig Trees) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> For every CNF formula φ with n variables, the depth D of the minimal Deligne-Lusztig tree associated with its incidence variety is linearly related to its resolution proof width w(φ), such that D = Θ(w(φ)).

Rationale (proposer's reasoning):

> Deligne-Lusztig trees provide a geometric representation theory framework for studying the action of finite reductive groups on certain algebraic varieties. Their depth could potentially encode structural information about the complexity of resolution proofs, revealing new insights into the nature of NP-completeness.

Taxonomy category: `Geometric_Representation_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9743b5c041bb6a97`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the minimal Deligne-Lusztig tree depth and the resolution proof width for all CNF formulas with n variables (n ≤ 40) exceeds 0.95, calculated across 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'Deligne-Lusztig trees' AND 'resolution proof complexity'` - `'incidence variety' AND 'Boolean satisfiability problems'` - `'linear relationship' AND 'depth of Deligne-Lusztig tree' AND 'proof width'`

Top relevant hits considered: - [http://arxiv.org/abs/1911.03412v2] On loop Deligne–Lusztig varieties of Coxeter-type for inner forms of GL_n - [http://arxiv.org/abs/0807.2786v2] On the connectedness of Deligne-Lusztig varieties - [http://arxiv.org/abs/2406.06430v3] Decomposition of higher Deligne-Lusztig representations - [http://arxiv.org/abs/2408.10977v2] A point-variety incidence theorem over finite fields, and its applications - [http://arxiv.org/abs/2011.06551v1] Efficient Solution of Boolean Satisfiability Problems with Digital MemComputing - [http://arxiv.org/abs/1109.2142v1] Generalizing Boolean Satisfiability III: Implementation - [http://arxiv.org/abs/1211.3784v1] P-alcoves and nonemptiness of affine Deligne-Lusztig varieties - [http://arxiv.org/abs/2606.03062v1] On $\mathbb{J}$-strata with Parahoric Stabilizers in Affine Deligne-Lusztig Varieties

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(10):  # Generate 10 clauses with n variables
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        stack = []
        seen = set()

        for literal in cnf:
            if literal not in seen and -literal not in seen:
                stack.append(literal)
                seen.add(literal)

        while stack:
            literal = stack.pop()
```

```
            if -literal in seen:
                continue
            seen.add(-literal)
            for clause in cnf:
                if literal in clause:
                    new_clause = [l for l in clause if l != literal]
                    if not new_clause:
                        return len(stack) + 1
                    for l in new_clause:
                        if -l not in seen:
                            stack.append(l)
        return len(stack)

    def deligne_lusztig_tree_depth(n):
        # Simplified version of Deligne-Lusztig tree depth calculation
        # This is a placeholder and should be replaced with actual computation
        return n

    n = random.randint(5, 40)
    cnf = generate_cnf(n)

    deligne_lusztig_depth = deligne_lusztig_tree_depth(n)
    resolution_width_value = resolution_width(cnf)

    return {
        "metric_name": "deligne_lusztig_tree_depth",
        "metric_value": deligne_lusztig_depth,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": deligne_lusztig_depth == resolution_width_value,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.95:
        RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std={math.sqrt(sum((x -
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        RESULT: FALSIFIED counterexample="{counterexample}" first_failing_seed={next(r["seed"] for
    else:
        RESULT: INCONCLUSIVE insufficient_support
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it did not complete within the allotted time limit. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15123
2	propose	ollama_remote	glm4:latest	0	0	23109
3	preregistration	ollama_remote	glm4:latest	0	0	12406
4	novelty	ollama_remote	glm4:latest	0	0	8636
5	novelty	ollama_remote	glm4:latest	0	0	9703
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13488
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8796
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10030
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9363
10	judge	ollama_remote	glm4:latest	0	0	15252

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 125906 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/43e22ea23a6f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/43e22ea23a6f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/43e22ea23a6f.tar.gz` (if generated)